Project Specification

Project Name: ExamLens - A Private Local VLM Agent That Turns Scanned Exam Papers into Ready-to-Teach Lecture Slides

Track: Track 2 - Private AI Agent Development and Local Deployment

1. Application Scenario

High-school teachers spend 3-5 hours after every exam retyping questions, writing explanations and building review slides. ExamLens automates this: the teacher drops the scanned pages of an exam paper into one folder and runs one command. The local agent (1) solves the paper, (2) transcribes all 23 questions verbatim together with their reading passages, and (3) renders a ready-to-teach PowerPoint deck containing the original question text, reference answers, step-by-step explanations, common mistakes and a full scanned-paper appendix. Exam papers are unpublished, sensitive material, therefore the whole pipeline runs 100% offline on a single AMD Radeon GPU in the Radeon Cloud and no cloud API is ever called.

2. Agent Architecture

ExamLens is a four-stage agent pipeline with a deterministic controller and file-based memory:

Scanned pages (PNG)

-> Stage 1 VLM Solver (vLLM on ROCm): reads pages, writes exam_result.txt

-> Stage 2 VLM Transcriber: one prompt per question, page-whitelist filter, writes explanations.json

-> Stage 3 Validator: JSON salvage parser, coverage check (23/23), auto re-run queue for missing items

-> Stage 4 Renderer (python-pptx): 40+ slide lecture deck

3. Core Capabilities

(1) Fully local agent on an AMD Radeon GPU, no network egress.

(2) Real-world scenario task: a complete Grade-10 Chinese final exam, 8 pages, 23 questions.

(3) Tool invocation: Pillow (image preprocessing), JSON repair parser, whitelist filter, python-pptx (slide rendering).

(4) Multi-step planning: solve -> transcribe -> validate -> re-run missing -> render; state saved after every step, crash-safe and idempotent.

(5) Stability engineering: temperature 0.1, bounded per-question outputs, two-attempt retries, anti-hallucination whitelist.

4. Model and Local Deployment

Open-source vision-language model (Qwen2.5-VL family), BF16 weights, served by vLLM with PagedAttention on one AMD Radeon GPU (ROCm build). HuggingFace Transformers and Pillow for preprocessing; python-pptx for rendering. Model weights stored on local disk.

5. GPU Optimization on AMD Radeon

- gpu_memory_utilization = 0.9 with PagedAttention for maximum KV-cache throughput.

- BF16 weights and vLLM graph capture (PIECEWISE / FULL) for fast decode.

- max_model_len tuned per stage (12k for full-page OCR, 32k for per-question passes) to fit the KV cache.

- Torch.compile / Dynamo cache reused across runs for warm starts.

- Measured on the Radeon GPU: about 2000 tokens/s prefill and about 30 tokens/s decode.